

VulnWatch

Portail de veille des vulnérabilités des solutions de cybersécurité — collecte, centralise et priorise automatiquement les CVE publiées sur les principales solutions de sécurité du marché (Palo Alto, Fortinet, Cisco, solutions antivirus, etc.).

Table of Contents

1. Présentation du projet	1
2. Installation pas à pas	2
3. Utilisation	3
4. Architecture	3
5. API Endpoints	4
6. Dépannage	5
7. Équipe	6

1. Présentation du projet

1.1 Contexte

Les équipes sécurité doivent assurer une veille permanente sur les vulnérabilités affectant les solutions de sécurité déployées (pare-feux, antivirus, EDR, etc.). Cette veille est aujourd'hui dispersée entre plusieurs sources officielles (NVD, bulletins PSIRT des éditeurs, catalogue CISA KEV), ce qui rend la consultation fastidieuse et la priorisation difficile.

1.2 Objectif

VulnWatch automatise l'ensemble du cycle : **collecte** → **normalisation** → **stockage** → **restitution**, afin qu'un analyste sécurité puisse consulter en un seul endroit toutes les vulnérabilités pertinentes, déjà triées par criticité et enrichies du statut d'exploitation active.

1.3 Fonctionnalités principales

- **Tableau de bord** : nombre de CVE par vendeur, par sévérité, évolution sur 14 jours
- **Liste filtrable** : par vendeur, par produit, par score CVSS (min/max), par statut KEV, recherche libre
- **Fiche détail CVE** : description complète, produits affectés, score CVSS, versions impactées, lien vers l'avis officiel, statut de patch
- **Système d'alertes** : notification automatique quand une nouvelle CVE critique apparaît, ou quand une CVE devient activement exploitée (CISA KEV)
- **Export** : CSV et PDF, avec les filtres actifs appliqués

- **Automatisation** : synchronisation quotidienne automatique (2h00 du matin) + déclenchement manuel possible
- **Journalisation** : historique de chaque synchronisation (succès/échec, nombre de CVE traitées)

1.4 Sources de données

Source	Rôle
NVD (NIST)	Source principale — score CVSS, description, références, date de publication
CISA KEV	Enrichissement — marque les CVE activement exploitées
Fortinet PSIRT (RSS)	Flux vendeur — CVE citées dans les bulletins officiels Fortinet
Palo Alto PSIRT (RSS)	Flux vendeur — CVE citées dans les bulletins officiels Palo Alto

2. Installation pas à pas

2.1 Prérequis

- Python 3.10 ou supérieur
- Un navigateur web
- Une connexion internet (pour la collecte de données réelles)

2.2 Cloner le projet

```
git clone https://github.com/darmoulmourchene-rgb/VulnWatch.git
cd VulnWatch
```

2.3 Installer les dépendances du backend

Dépendances principales : fastapi, uvicorn, sqlalchemy, apscheduler, requests, feedparser, reportlab.

Installez manuellement :

```
pip install fastapi uvicorn sqlalchemy apscheduler requests feedparser reportlab
```

2.4 Première collecte de données

```
python collect_data.py
```

Ce script interroge NVD, CISA KEV et les flux PSIRT, puis crée automatiquement la base de données SQLite (`vulnwatch.db`) si elle n'existe pas. Comptez 2 à 5 minutes (le script respecte volontairement la limite de débit de l'API NVD).

⚠ **Si vous modifiez `Models.py` par la suite** (ajout d'un champ ou d'une table), supprimez `vulnwatch.db` avant de relancer — SQLAlchemy ne migre pas automatiquement une base existante.

2.5 Lancer l'API backend

```
python api_fast.py
```

L'API démarre sur `http://127.0.0.1:8000`

2.6 Lancer le frontend

Depuis le dossier frontend, deux options :

Option A — Extension Live Server (VS Code, recommandé) Clic droit sur `index.html` → *Open with Live Server*

Option B — Serveur Python

```
cd frontend
python -m http.server 8001
```

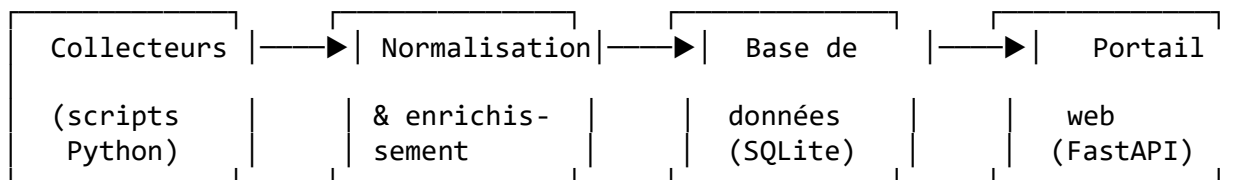
Puis ouvrir `http://localhost:8001`.

3. Utilisation

1. **Dashboard** : vue d'ensemble à l'ouverture — indicateurs chiffrés, graphique d'évolution sur 14 jours, répartition par criticité et par éditeur, liste des CVE critiques récentes.
2. **Vulnérabilités** : liste complète, filtrable par éditeur, produit, sévérité, plage de score CVSS, et statut KEV. Cliquer sur une ligne ouvre le détail complet.
3. **Recherche** : la barre de recherche en haut filtre par ID CVE, produit, éditeur ou mot-clé dans la description.
4. **Alertes** : dans la barre supérieure affiche les alertes non lues (nouvelle CVE critique ou nouvellement exploitée). Cliquer sur une alerte l'ouvre et la marque comme lue.
5. **Export** : les boutons CSV et PDF dans la vue Liste exportent les résultats actuellement filtrés.
6. **Rafraîchissement** : recharge les données affichées depuis l'API. La collecte complète depuis les sources externes tourne automatiquement chaque nuit à 2h00

4. Architecture

Le projet suit l'architecture en couches classique définie dans la fiche de stage :



Couche	Rôle	Fichier(s)
Collecte	Récupère la donnée brute depuis NVD, CISA KEV, et les flux RSS PSIRT	<code>collect_data.py</code>

Couche	Rôle	Fichier(s)
Traitement	Normalise vers un schéma unique, calcule la sévérité, enrichit avec le statut KEV	<code>collect_data.py</code> (fonctions <code>normalize_nvd_item</code> , <code>extract_cvss</code> , <code>enrich_with_kev</code>)
Persistance	Stocke les vulnérabilités, alertes et journaux de synchronisation	<code>Models.py</code> , <code>Database.py</code>
Présentation	Expose une API REST filtrable et sert l'interface web	<code>api_fast.py</code> (backend), <code>index.html</code> / <code>detail.html</code> / <code>script.js</code> (frontend)

4.1 Schéma de données (table `vulnerabilities`)

Champ	Type	Description
<code>id</code>	String (clé primaire)	Identifiant CVE, ex: CVE-2024-3400
<code>vendor</code>	String	Éditeur concerné
<code>product</code>	String	Produit principal (rétrocompatibilité)
<code>products</code>	Text (JSON)	Liste de tous les produits affectés
<code>description</code>	Text	Description de la vulnérabilité
<code>cvss_score</code>	Float	Score CVSS (0–10)
<code>severity</code>	String	CRITICAL / HIGH / MEDIUM / LOW / UNSCORED
<code>unscored</code>	Boolean	true si NVD n'a pas encore attribué de score
<code>published_date</code>	String (ISO)	Date de publication
<code>patch_available</code>	Boolean	Correctif disponible ou non
<code>kev_status</code>	Boolean	Présente dans le catalogue CISA KEV
<code>versions</code>	String	Versions affectées
<code>references</code>	Text (JSON)	Liens vers les avis officiels
<code>source</code>	String	Origine ("NVD", "NVD + Fortinet PSIRT", ...)

4.2 Tables complémentaires

- **alerts** : une ligne par alerte générée (nouvelle CVE critique ou nouvellement KEV), avec anti-doublon par CVE + motif.
- **sync_logs** : historique des collectes (début, fin, statut, nombre de CVE traitées, message d'erreur éventuel).

5. API Endpoints

Méthode	Route	Description
GET	/	Informations générales sur l'API
GET	/api/health	Statut de l'API et de la base de données
GET	/api/vulnerabilities	Liste filtrable (<code>vendor</code> , <code>product</code> , <code>severity</code> , <code>kev</code> , <code>min_score</code> , <code>max_score</code> , <code>search</code> , <code>limit</code> , <code>offset</code>)

Méthode	Route	Description
GET	/api/vulnerabilities/{cve_id}	Détail complet d'une CVE
GET	/api/stats	Statistiques agrégées (total, par sévérité, par éditeur, 30 derniers jours)
GET	/api/vendors	Liste des éditeurs suivis
GET	/api/products	Liste des produits, filtrable par éditeur
GET	/api/export/csv	Export CSV (respecte les filtres actifs)
GET	/api/export/pdf	Export PDF (respecte les filtres actifs)
GET	/api/alerts	Liste des alertes (filtrable par statut lu/non lu)
POST	/api/alerts/{id}/ack	Marque une alerte comme lue
POST	/api/alerts/ack_all	Marque toutes les alertes comme lues
POST	/api/sync	Déclenche une collecte complète immédiate
GET	/api/sync-logs	Historique des synchronisations

La documentation interactive Swagger est également disponible sur <http://127.0.0.1:8000/docs> une fois l'API lancée.

6. Dépannage

“Le site n’affiche rien / les scores sont tous à 0”

Vérifiez d'abord l'API directement :

<http://127.0.0.1:8000/api/vulnerabilities?limit=3>. Si les données y sont correctes, le problème vient du cache navigateur — faites Ctrl+Shift+R (rechargement force)

“no such column” après une modification de `Models.py`

SQLAlchemy ne migre pas une base existante. Supprimez `vulnwatch.db` et relancez `collect_data.py`.

Certaines CVE affichent “N/A” au lieu d’un score

Normal : NVD n'a pas encore terminé l'analyse CVSS de cette CVE (statut *Awaiting Analysis*), généralement pour des CVE très récentes. Ce n'est pas une erreur — le champ `unscored` distingue ce cas d'un score réellement nul.

Un flux RSS PSIRT ne retourne aucune CVE

Les vendeurs changent parfois l'URL de leurs flux RSS. Vérifiez que `FORTINET_PSIRT_RSS` et `PALOALTO_PSIRT_RSS` dans `collect_data.py` sont toujours valides.

La collecte automatique de nuit ne semble pas avoir tourné

Consultez `GET /api/sync-logs` pour voir l'historique complet, y compris les échecs et leur message d'erreur.

Import errors (`ModuleNotFoundError: Models / models`)

Sous Windows, les imports Python sont sensibles à la casse selon l'éditeur utilisé. Assurez-vous que les noms de fichiers (`Models.py`, `Database.py`) correspondent exactement aux instructions `import` dans `api_fast.py` et `collect_data.py`.

7. Équipe

Projet réalisé en binôme dans le cadre d'un stage — Nourchène et Roua .